# EC2 Deployment Documentation – Portfolio Project

**********

## 1. EC2 Instance Creation

First, I created an EC2 instance with the following configuration:

- **Name**: portfolio-nishant
- **AMI**: Ubuntu
- **Instance Type**: t3.small (suitable for small applications)
- **Key Pair**: nishant-hp-key (used for secure SSH login)
- **Security Group Ports**:
    - 22 → SSH access
    - 80 → HTTP (website access)
    - 443 → HTTPS (secure website access)

After configuring everything, I launched the EC2 instance.

---

## 2. Login to EC2 Instance

ssh ubuntu@34.207.224.235

This command connects to the EC2 server using SSH.

- `ubuntu` is the default user
- IP address is the public IP of the instance

## What is Nginx?

Nginx is software that receives user requests from the internet and sends them to your application (like Node.js), then returns the response back to the user.

## 3. Update System Packages

```
sudo apt update
```

This updates the package list so the system knows about the latest available software versions.

---

# 4. Install Required Packages

```
sudo apt install curl gnupg2 ca-certificates lsb-release ubuntu-keyring
```

These packages are required for:

- Secure downloads (`curl`)
- Key management (`gnupg2`)
- OS information (`lsb-release`)
- Certificate validation

---

## 5. Install Nginx (Official Repository)

### Add Nginx Signing Key

```
curl https://nginx.org/keys/nginx_signing.key | gpg --dearmor | sudo tee /usr/share/keyrings/nginx-archive-keyring.gpg >/dev/null
```

This downloads and stores the official Nginx security key.

---

### Verify Key

```
gpg --dry-run --quiet --no-keyring --import --import-options import-show /usr/share/keyrings/nginx-archive-keyring.gpg
```

This checks if the key is valid and correctly added.

---

### Add Nginx Repository

```
echo "deb [signed-by=/usr/share/keyrings/nginx-archive-keyring.gpg] https://nginx.org/packages/ubuntu `lsb_release -cs` nginx" | sudo tee /etc/apt/sources.list.d/nginx.list
```

Adds the official Nginx repository to install the latest version.

---

### Set Priority

```
echo -e "Package: *\nPin: origin nginx.org\nPin: release o=nginx\nPin-Priority: 900\n" | sudo tee /etc/apt/preferences.d/99nginx
```

Ensures the system prefers Nginx packages from the official repo.

### Install Nginx

```
sudo apt update
sudo apt install nginx
```

Installs the Nginx web server.

---

### Check Version

```
nginx -v
```

Displays installed Nginx version.

---

### Start and Enable Nginx

```
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl status nginx
```

- Start → runs Nginx
- Enable → auto-start on reboot
- Status → check if running

---

## What is Node.js

Node.js is a runtime environment that allows you to run JavaScript code on the server (backend).

## What is npm

npm is a tool used to install, manage, and run JavaScript packages (libraries) in a Node.js project

## 6. Install Node.js

```
curl -fsSL https://deb.nodesource.com/setup_current.x | sudo -E bash -
sudo apt install -y nodejs
```

This installs the latest Node.js version.

---

### Verify Installation

```
node -v
npm -v
```

Checks Node.js and npm versions.

---

## What is PM2

PM2 is a tool that keeps your Node.js app running in the background and automatically restarts it if it stops or crashes.

## 7. Install PM2 (Process Manager)

```
sudo npm install -g pm2
```

PM2 is used to run Node.js apps in the background.

```
pm2 -v
```

Verifies installation.

---

## 8. Project Setup

```
cd portfolio/
npm install
```

- `cd portfolio/` → go to project folder
- `npm install` → installs dependencies

---

## 9. Run Application (Development Mode)

```
npm run dev
```

Starts the Next.js app on port 3000.

---

## Why do we use a Reverse Proxy (like Nginx)

A reverse proxy sits between users and your backend server to handle requests more efficiently, securely, and cleanly.

## 10. Configure Nginx as Reverse Proxy

```
sudo vim /etc/nginx/conf.d/default.conf
```

Edit Nginx configuration:

```
server {
    listen 80;
    server_name vishnugaur.in www.vishnugaur.in;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;

        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";

        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

This forwards incoming traffic from port 80 to the Node.js app running on port 3000.

---

### Test and Reload Nginx

```
sudo nginx -t
sudo systemctl reload nginx
sudo systemctl restart nginx
```

```
sudo systemctl status nginx
```

- Test → checks config syntax
- Reload → apply changes
- Restart → fully restart service

---

## 11. Setup SSL (HTTPS)

```
sudo apt install certbot python3-certbot-nginx -y
```

Installs Certbot for SSL.

```
sudo certbot --nginx -d vishnugaur.in -d www.vishnugaur.in
```

This:

- Generates SSL certificate
- Configures HTTPS automatically

---

## 12. Run App in Background (Production)

```
pm2 start npm --name "portfolio-dev" -- run dev
```

Runs the app in background using PM2.

---

### Check Running Apps

```
pm2 list
```

Shows all running processes.

---

## 13. Enable Auto Start on Reboot

```
pm2 startup
```

This generates a command:

```
sudo env PATH=$PATH:/usr/bin pm2 startup systemd -u ubuntu --hp /home/ubuntu
```

Run that command.

Then save current processes:

```
pm2 save
```

---

## 15. Access Application

https://vishnugaur.in

---

## 14. Final Result

Now your application is:

- Running in background (PM2)
- Accessible via domain
- Secured with HTTPS
- Automatically starts on reboot

---

### Conclusion

This setup ensures that the Next.js application is deployed properly on EC2 with Nginx as a reverse proxy and PM2 for process management. It also includes SSL security using Certbot, making the application production-ready.